

Web Engineering & Development (SWE 363)

# MongoDB

**Dr. Khadijah Al Safwan**

## In today's lecture:

- Creating RESTful Web APIs (Node)
- Using RESTful Web APIs with Fetch

## Reference:

- Zybook: 7.3, 7.4

# Creating RESTful Web APIs (Node)

# What is REST?

**REST** = **RE**presentational **S**tate **T**ransfer

A set of principles for designing web APIs that are:

- **Simple** - Easy to understand and use
- **Standard** - Uses HTTP methods consistently
- **Scalable** - Can grow with your application
- **Stateless** - Each request is independent

# What is a RESTful API?

A **RESTful API** is an interface that follows REST principles

## Key characteristics:

- Uses **HTTP methods** (GET, POST, PUT, DELETE)
- Resources identified by **URLs**
- Returns data in **JSON format**
- **Stateless** – no session stored on server

# What is a RESTful API?

A **RESTful API** is an interface that follows REST principles

## Example:

|        |                |                           |
|--------|----------------|---------------------------|
| GET    | /api/songs     | → Get all songs           |
| POST   | /api/songs     | → Create a new song       |
| GET    | /api/songs/123 | → Get song with ID 123    |
| PUT    | /api/songs/123 | → Update song with ID 123 |
| DELETE | /api/songs/123 | → Delete song with ID 123 |

# Why RESTful APIs?

## Benefits:

- **Standardized** – Everyone uses the same patterns
- **Language-independent** – Works with any programming language
- **Easy to understand** – Clear URL structure
- **Scalable** – Can handle many clients
- **Cacheable** – Responses can be cached

## Real-world examples:

- Twitter API, GitHub API, Weather APIs
- Your own backend for your web application

# CRUD Operations

**CRUD** = **C**reate, **R**ead, **U**ppdate, **D**eleate

These are the four basic operations for managing data

| Operation | HTTP Method | Purpose              |
|-----------|-------------|----------------------|
| Create    | POST        | Add new data         |
| Read      | GET         | Retrieve data        |
| Update    | PUT         | Modify existing data |
| Delete    | DELETE      | Remove data          |



# HTTP Methods for CRUD

```
// POST – Create a new resource
POST /api/songs
Body: { "title": "Song Title", "artist": "Artist Name" }

// GET – Read/Retrieve resources
GET /api/songs          → Get all songs
GET /api/songs/123      → Get song with ID 123

// PUT – Update an existing resource
PUT /api/songs/123
Body: { "title": "Updated Title" }

// DELETE – Delete a resource
DELETE /api/songs/123   → Delete song with ID 123
```

# Setting Up Express for RESTful APIs

## Required packages:

```
npm install express cors mongoose dotenv
```

## Basic server setup:

```
import express from "express";
import cors from "cors";

const app = express();
const PORT = 3000;
// Middleware
app.use(cors()); // Allow front-end to connect
app.use(express.json()); // Parse JSON request bodies
app.listen(PORT, () => { console.log(`Server running on port ${PORT}`); });
```

# Why `express.json()` is Important

## Without `express.json()`:

- `req.body` will be `undefined`
- Cannot read POST/PUT data

## With `express.json()`:

- Automatically parses JSON from request body
- Makes data available in `req.body`

# Connecting to MongoDB

## Using Mongoose to connect:

```
import mongoose from "mongoose";
import dotenv from "dotenv";

dotenv.config();

mongoose.connect(process.env.MONGO_URL)
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error("Connection error:", err.message));
```

## In .env file:

```
MONGO_URL=mongodb+srv://username:password@cluster.mongodb.net/dbname
```

# Creating a Schema and Model

## Define the data structure:

```
import mongoose from "mongoose";

const songSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  artist: { type: String, required: true, trim: true },
  year: { type: Number, min: 1900, max: 2100 }
}, { timestamps: true });

const Song = mongoose.model("Song", songSchema);
export default Song;
```

**What this does:** Defines fields and their types, adds validation rules, and automatically adds `createdAt` and `updatedAt` timestamps

# Creating Routes: POST (Create)

## Create a new song:

```
app.post("/api/songs", async (req, res) => {  
  try {  
    const { title = "", artist = "", year } = req.body || {};  
  
    const created = await Song.create({  
      title: title.trim(),  
      artist: artist.trim(),  
      year  
    });  
  
    res.status(201).json(created);  
  } catch (err) {  
    res.status(400).json({ message: err.message || "Create failed" });  
  }  
});
```

# Creating Routes: GET (Read All)

## Get all songs:

```
app.get("/api/songs", async (req, res) => {  
  try {  
    const songs = await Song.find().sort({ createdAt: -1 });  
    res.json(songs);  
  } catch (err) {  
    res.status(500).json({ message: err.message });  
  }  
});
```

## What this does:

- `Song.find()` - Gets all documents
- `.sort({ createdAt: -1 })` - Newest first
- Returns array of all songs

# Creating Routes: GET (Read One)

## Get a single song by ID:

```
app.get("/api/songs/:id", async (req, res) => {  
  try {  
    const song = await Song.findById(req.params.id);  
  
    if (!song) {  
      return res.status(404).json({ message: "Song not found" });  
    }  
  
    res.json(song);  
  } catch (err) {  
    res.status(500).json({ message: err.message });  
  }  
});
```



# Creating Routes: PUT (Update)

## Update an existing song:

```
app.put("/api/songs/:id", async (req, res) => {
  try {
    const updated = await Song.findByIdAndUpdate(
      req.params.id,
      req.body || {},
      { new: true, runValidators: true } // Returns updated document and validates the update
    );
    if (!updated) {
      return res.status(404).json({ message: "Song not found" });
    }
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message || "Update failed" });
  }
});
```

# Creating Routes: DELETE

## Delete a song:

```
app.delete("/api/songs/:id", async (req, res) => {
  try {
    const deleted = await Song.findByIdAndDelete(req.params.id);

    if (!deleted) {
      // 404 - Not Found (song doesn't exist)
      return res.status(404).json({ message: "Song not found" });
    }

    res.status(204).end(); // 204 - No content, successful deletion
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

# HTTP Status Codes

## Common status codes for RESTful APIs:

| Code | Meaning      | Use Case               |
|------|--------------|------------------------|
| 200  | OK           | Successful GET, PUT    |
| 201  | Created      | Successful POST        |
| 204  | No Content   | Successful DELETE      |
| 400  | Bad Request  | Validation error       |
| 404  | Not Found    | Resource doesn't exist |
| 500  | Server Error | Server-side error      |

# Error Handling Best Practices

## Always handle errors:

```
app.post("/api/songs", async (req, res) => {  
  try {  
    // Your code here  
  
  } catch (err) {  
    // Handle validation errors  
    if (err.name === "ValidationError") {  
      return res.status(400).json({ message: err.message });  
    }  
    // Handle other errors  
    res.status(500).json({ message: "Server error" });  
  }  
});
```

# Error Handling Best Practices

## Why it matters:

- Prevents server crashes
- Provides helpful error messages
- Better user experience

# RESTful API Best Practices

## 1. Use consistent URL patterns:

|                                |                   |
|--------------------------------|-------------------|
| <code>/api/resource</code>     | → Collection      |
| <code>/api/resource/:id</code> | → Single resource |

## 2. Use appropriate HTTP methods:

- GET for reading
- POST for creating
- PUT for updating
- DELETE for deleting

# RESTful API Best Practices

## 3. Return proper status codes:

- 200/201 for success
- 400 for client errors
- 404 for not found
- 500 for server errors

## 4. Always validate input data

## 5. Use meaningful error messages

# Using RESTful Web APIs with Fetch



# What is the Fetch API?

**Fetch API** is a built-in JavaScript function for making HTTP requests

## Why use Fetch?

- Built into modern browsers
- Works with async/await
- Returns Promises
- Simple and clean syntax

## Before Fetch:

- Had to use XMLHttpRequest (complex)
- Required lots of code

## With Fetch:

- Simple one-line requests
- Easy to read and write

# Basic Fetch Syntax

## Simple GET request:

```
fetch('http://localhost:3000/api/songs')  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  })  
  .catch(error => {  
    console.error('Error:', error);  
  });
```

## What happens:

Send request to URL -> Wait for response -> Convert response to JSON -> Use the data

# Fetch: GET Request

## Get all songs:

```
async function getAllSongs() {
  try {
    const response = await fetch('http://localhost:3000/api/songs');

    // Always check if the response is ok
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    const songs = await response.json();
    return songs;
  } catch (error) {
    ...
  }
}
```

# Fetch: POST Request (Create)

## Create a new song:

```
async function createSong(songData) {  
  try {  
    const response = await fetch('http://localhost:3000/api/songs', {  
      method: 'POST', // POST request to create a new song  
      headers: {  
        'Content-Type': 'application/json', // Set the content type to JSON  
      },  
      body: JSON.stringify(songData) // Convert the song data to JSON  
    });  
    if (!response.ok) {  
      ...  
    }  
    const created = await response.json();  
    return created;  
  } catch (error) {  
    ...  
  }  
}
```

# Fetch: PUT Request (Update)

## Update an existing song:

```
async function updateSong(id, songData) {  
  try {  
    const response = await fetch(`http://localhost:3000/api/songs/${id}`, {  
      method: 'PUT',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify(songData)  
    });  
  
    if (!response.ok) { ... }  
    const updated = await response.json();  
    return updated;  
  } catch (error) { ... }  
}
```

# Fetch: DELETE Request

## Delete a song:

```
async function deleteSong(id) {  
  try {  
    const response = await fetch(`http://localhost:3000/api/songs/${id}`, {  
      method: 'DELETE'  
    });  
  
    if (!response.ok) { ... }  
  
    // DELETE usually returns 204 (No Content)  
    if (response.status === 204) {  
      return { success: true };  
    }  
    return await response.json();  
  } catch (error) { ... }  
}
```

# Complete Flow: Front-end to Back-end

## 1. User action in React:

```
const handleSubmit = async (formData) => {  
  await apiCreateSong(formData);  
};
```

## 2. Fetch sends request:

```
fetch('http://localhost:3000/api/songs', {  
  method: 'POST',  
  body: JSON.stringify(formData)  
});
```

# Complete Flow: Front-end to Back-end

## 3. Express receives request:

```
app.post('/api/songs', async (req, res) => {  
  const created = await Song.create(req.body);  
  res.status(201).json(created);  
});
```

## 4. Response sent back:

```
// Front-end receives response  
const created = await response.json();
```



# Testing RESTful APIs

## Tools for testing:

- **Postman** - Popular API testing tool
- **Browser** - For GET requests
- **curl** - Command-line tool
- **Your front-end** - Real-world testing

# Testing RESTful APIs

## Example with curl:

```
# GET all songs
curl http://localhost:3000/api/songs

# POST a new song
curl -X POST http://localhost:3000/api/songs \
  -H "Content-Type: application/json" \
  -d '{"title":"My Song","artist":"Artist Name","year":2024}'
```

30m

# Demo

<https://classroom.github.com/a/b5Rjx0N4>